

大模型基础面试题总结

很多同学准备大模型面试时，第一反应是去背 Transformer、Attention、RLHF 这些词。不是说这些不重要，但对大部分后端转 AI 应用开发、AI 工程应用岗位来说，面试官更关心的是另一件事：

你是不是真的理解大模型调用链路里的工程约束。

比如 Token 为什么会影响成本和延迟？上下文窗口为什么不是越大越好？Temperature 为什么会影响结构化输出稳定性？Function Calling 为什么不能让模型直接执行真实业务操作？这些问题看起来基础，答不好就会暴露一个信号：你可能只是调过 API，还没有把大模型当作生产系统里的一个不稳定外部依赖来治理。

这份大模型基础面试题主要根据 AI 专栏现有文章整理。它不是让你机械刷题，而是帮你建立一条复习主线：

1. 先理解 **Token**、**上下文窗口**、**采样参数**，知道模型为什么会不稳定。
2. 再理解 **API 调用工程**，知道一次模型调用在生产里要经过哪些治理环节。
3. 接着理解**结构化输出与工具调用**，知道怎么让模型输出能被程序消费。
4. 最后理解 **AI 应用评测**，知道怎么判断你的 AI 应用到底有没有变好。

面试官真正想考什么

大模型基础题表面上问概念，实际考的是工程判断。你可以按下面这张表来理解。

考察方向	面试官想确认什么	常见扣分点
Token 和上下文	你是否理解成本、延迟、窗口限制和信息取舍	只说 Token 是“词元”，讲不出工程影响
采样参数	你是否知道如何在创造性和稳定性之间取舍	把 Temperature 说成越高越聪明
API 调用链路	你是否具备把模型接入生产系统的经验	只说调用 HTTP 接口，忽略重试、限流、幂等
结构化输出	你是否知道自然语言约束不等于工程契约	认为“请返回 JSON”就足够可靠
评测闭环	你是否能验证效果，而不是凭感觉调 Prompt	只看公开 benchmark，不做业务 Golden Set

一个不错的回答通常不是定义式的，而是“概念 + 问题 + 工程解法”。例如问 Token，你可以先解释 Token 是模型处理文本的基本单位，再补一句：Token 直接影响上下文容量、推理成本、响应延迟和截断风险，所以生产系统里要做预算估算、历史消息压缩、RAG 证据筛选和最大输出限制。

这就比单纯背定义强很多。

LLM 运行机制

参考文章：《LLM 运行机制：Token、上下文窗口与采样参数怎么影响输出》

这一组题是大模型面试的地基。不要只记术语，要重点理解这些概念如何影响真实系统的稳定性、成本和答案质量。建议掌握这些关键点：

- Token 不是字符，也不是中文里的“字”。不同语言、符号、代码片段的切分方式不同，因此同样长度的中文、英文、代码，Token 消耗可能差很多。
- 上下文窗口不是无限记忆。窗口越大，成本、延迟、噪声、Lost in the Middle 风险都会增加。

- Temperature、Top-P、Top-K 控制的是采样分布，不是模型“智商”。生产环境通常更关注稳定性和可复现性。

- 幻觉不是单靠某个参数就能消灭的。更可靠的做法是 RAG、工具调用、引用来源、输出校验和评测闭环一起做。

高频面试题：

- Token 是什么？为什么中文、英文、代码消耗的 Token 不一样？

- 上下文窗口是什么？上下文窗口越大，效果一定越好吗？

- 什么是 Lost in the Middle 问题？长上下文场景下怎么缓解？

- Temperature、Top-P、Top-K 分别控制什么？生产环境怎么设置更稳？

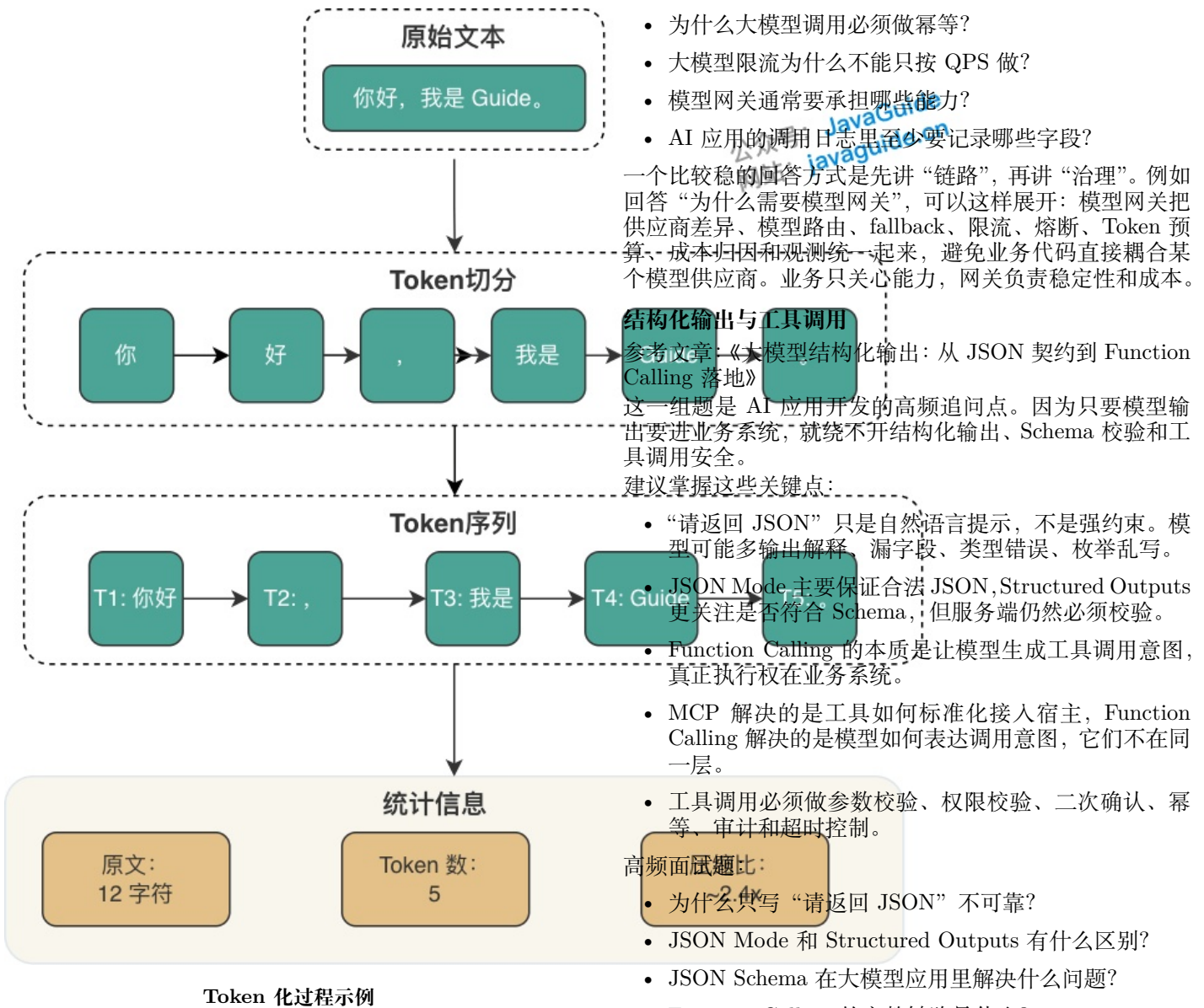
- 为什么 Temperature 设置为 0，模型输出仍然可能不完全一致？

- 大模型为什么会产生幻觉？常见缓解方案有哪些？

- Token 预算怎么估算？输入、输出、历史消息、RAG 证据如何取舍？

- 长上下文窗口会不会取代 RAG？二者分别适合什么场景？

面试追问通常会落到场景上。比如“你们的客服机器人历史会话太长怎么办？”这时不要只说“做摘要”，更完整的回答是：先区分必须保留的业务状态、最近对话、用户画像和可丢弃闲聊；再做 Token 预算；超过阈值时对历史消息做结构化摘要；RAG 证据只放最相关片段；最后通过评测集验证压缩后是否影响关键问题回答。



API 调用工程

参考文章:《大模型 API 调用工程实践: 流式输出、重试、限流与结构化返回》

这一组题考的是你有没有把模型当作生产依赖来治理。大模型 API 和普通 HTTP API 很像, 但又更麻烦: 它慢、贵、不稳定、输出不可完全控, 还可能被供应商限流。

建议掌握这些关键点:

- 一次模型调用不只是“发请求拿结果”, 而是一条完整链路: 请求校验、Prompt 组装、上下文注入、模型路由、限流、超时、重试、流式返回、结构化解析、日志和评测。
- Streaming 主要改善首字体验, 不等于减少总耗时, 也不等于降低 Token 成本。
- 重试必须和幂等绑定。没有幂等设计, 重试可能造成重复扣费、重复落库、重复执行工具。
- 限流不能只看 QPS, 还要看 RPM、TPM、并发数、上下文大小、最大输出和租户预算。

高频面试题:

- 大模型 API 调用的完整链路是什么?
- Streaming 为什么能改善用户体验? 它能减少总耗时和 Token 成本吗?
- SSE、WebSocket、HTTP Chunked 在流式输出场景下怎么选?
- 哪些大模型 API 错误可以重试? 哪些错误不能重试?

- 为什么大模型调用必须做幂等?
- 大模型限流为什么不能只按 QPS 做?
- 模型网关通常要承担哪些能力?
- AI 应用的调用日志里至少要记录哪些字段?

一个比较稳的回答方式是先讲“链路”, 再讲“治理”。例如回答“为什么需要模型网关”, 可以这样展开: 模型网关把供应商差异、模型路由、fallback、限流、熔断、Token 预算、成本归因和观测统一起来, 避免业务代码直接耦合某个模型供应商。业务只关心能力, 网关负责稳定性和成本。

结构化输出与工具调用

参考文章:《大模型结构化输出: 从 JSON 契约到 Function Calling 落地》

这一组题是 AI 应用开发的高频追问点。因为只要模型输出要进业务系统, 就绕不开结构化输出、Schema 校验和工具调用安全。

建议掌握这些关键点:

- “请返回 JSON”只是自然语言提示, 不是强约束。模型可能多输出解释、漏字段、类型错误、枚举乱写。
- JSON Mode 主要保证合法 JSON, Structured Outputs 更关注是否符合 Schema, 但服务端仍然必须校验。
- Function Calling 的本质是让模型生成工具调用意图, 真正执行权在业务系统。
- MCP 解决的是工具如何标准化接入宿主, Function Calling 解决的是模型如何表达调用意图, 它们不在同一层。
- 工具调用必须做参数校验、权限校验、二次确认、幂等、审计和超时控制。

高频面试题比:

- 为什么只写“请返回 JSON”不可靠?
- JSON Mode 和 Structured Outputs 有什么区别?
- JSON Schema 在大模型应用里解决什么问题?
- Function Calling 的完整链路是什么?
- Function Calling 和 MCP 有什么区别?
- MCP Tool 和普通 HTTP API 有什么关系?
- Agent Skill 和 Function Calling 是一回事吗?
- 结构化输出失败后怎么处理?
- 工具调用为什么必须做安全治理?
- 面试里怎么一句话概括结构化输出?

这类题最容易答得太抽象。建议始终带一个业务例子: 比如“退款工具调用”。模型可以生成 `refundOrder(orderId, amount, reason)` 的调用参数, 但后端必须确认当前用户是否有权限、订单是否属于本人、金额是否可退、是否已经退过、是否需要二次确认。模型只能提出意图, 不能绕过业务规则。

AI 应用评测

参考文章:《AI 应用评测体系: 从 Golden Set 构建到线上灰度闭环》

很多候选人会调 Prompt, 但说不清“怎么证明调得更好了”。这就是评测题的价值。面试官问评测, 通常是在判断你有没有生产意识。

建议掌握这些关键点:

- 公开 benchmark 只能粗略判断模型通用能力, 不能代表你的业务数据分布。
- Golden Set 的价值不在数量, 而在分布。正常路径、边缘场景、对抗样本、高权重失败都要覆盖。

- LLM-as-Judge 可以提高评测效率，但有位置偏差、冗长偏差、同源偏差和推理能力边界，不能完全替代人工。
- RAG 和 Agent 都要分段评测。只看最终答案，很难定位问题来自检索、生成、工具调用还是执行轨迹。

高频面试题：

- 为什么不能只靠公开 benchmark 评估 AI 应用质量？
- Golden Set 应该怎么构建？冷启动阶段没有生产日志怎么办？
- LLM-as-Judge 有哪些主要偏差？怎么缓解？
- RAG 评测为什么必须分检索和生成两段？
- Agent 评测为什么比普通问答和 RAG 更复杂？
- 离线评测、Trace 回放、线上灰度分别解决什么问题？
- CI 里的 AI 评测如何平衡速度和覆盖度？
- 如果 LLM-as-Judge 和人工评测结果不一致，应该怎么处理？

回答评测题时，尽量形成闭环：先有 Golden Set 做离线回归，再用 Trace 回放覆盖真实线上路径，最后通过灰度和线上采样验证真实用户分布。没有这条链路，优化基本靠感觉。

答题框架

大模型基础题可以套用简单框架：

1. 先解释概念：用一句话说清楚它是什么。
2. 再说明影响：它会影响质量、成本、延迟、稳定性还是安全。
3. 接着给工程做法：生产里如何配置、校验、降级或观测。
4. 最后补充边界：在哪些场景下会失效，或者需要和其他方案组合。

比如问“长上下文会不会取代 RAG”，可以这样答：

长上下文能提升单次输入容量，适合少量文档的深度分析，但它不能完全取代 RAG。企业知识库通常有海量文档、权限隔离、频繁更新、成本控制和引用溯源要求，不可能每次把所有内容塞进窗口。更现实的做法是用 RAG 做候选证据筛选，再把少量高质量上下文交给长上下文模型处理。

常见扣分点

- 只背定义，不讲工程影响。
- 把大模型 API 当普通 HTTP 接口，没有限流、重试、幂等、观测意识。
- 认为结构化输出等于“让模型返回 JSON”，忽略 Schema 和服务端校验。
- 认为 Function Calling 是模型直接执行函数，忽略业务系统的执行权和安全边界。
- 只看模型排行榜，不知道 Golden Set、Trace 回放和线上灰度。

复习建议

如果时间有限，建议按这个顺序复习：

1. 先看 Token、上下文窗口、采样参数，建立基础认知。
2. 再看 API 调用工程，理解从 Demo 到生产的差距。
3. 接着看结构化输出和 Function Calling，这是 AI 应用开发的高频追问点。
4. 最后看评测体系，尤其是 Golden Set、LLM-as-Judge、Trace 回放。

复习时不要只问自己“这个概念是什么”，还要继续追问三句：生产里会出什么问题？怎么定位？怎么治理？能答到这个层次，大模型基础面试基本就稳了。